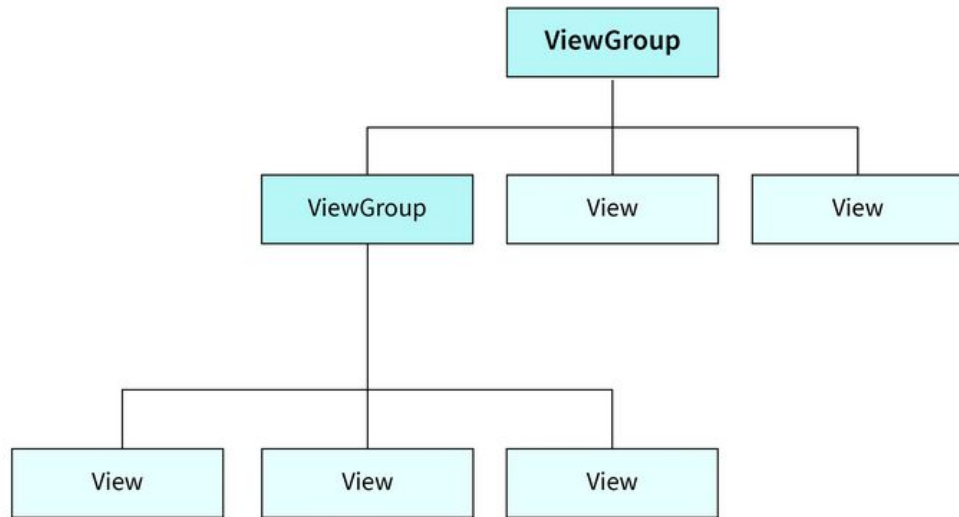


Layouts

Ms. Shital Dinde

Basic Terms

- View
 - Something the user can see or interact with
 - Different Input Controls
 - Button, DatePicker, EditText etc.
- ViewGroup
 - It is invisible container (group of different views)
 - Defines the layout structure(arrangement) for view and other viewgroups
- Layout
 - Layout defines the structure for a User Interface in your app activities
 - All elements in layout are built using a hierarchy of views and viewgroups
 - Arrangement of views and view groups for building UI of Activity



Different Layouts

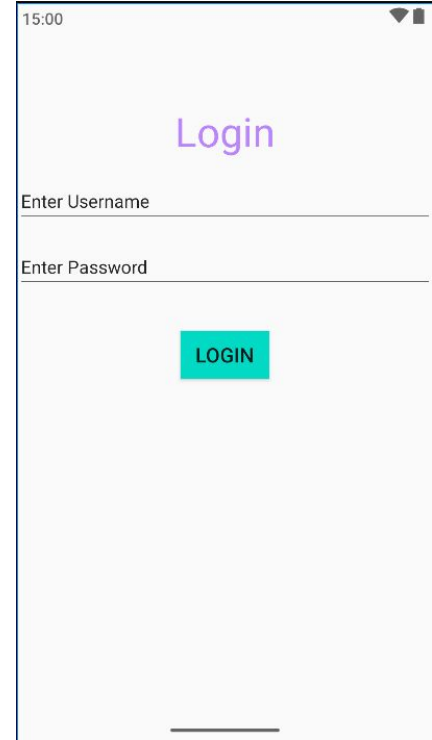
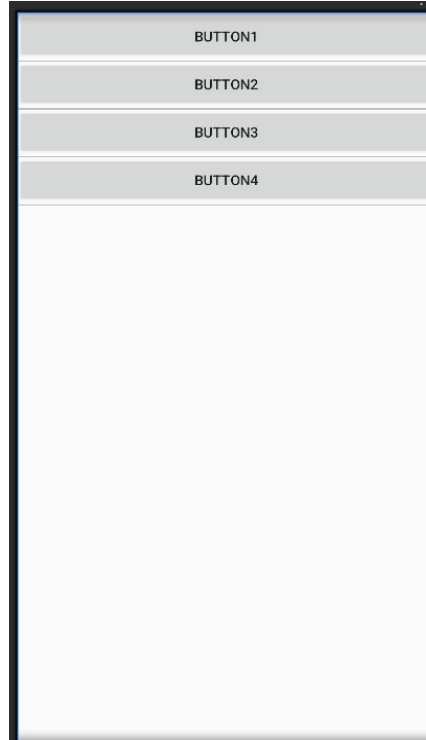
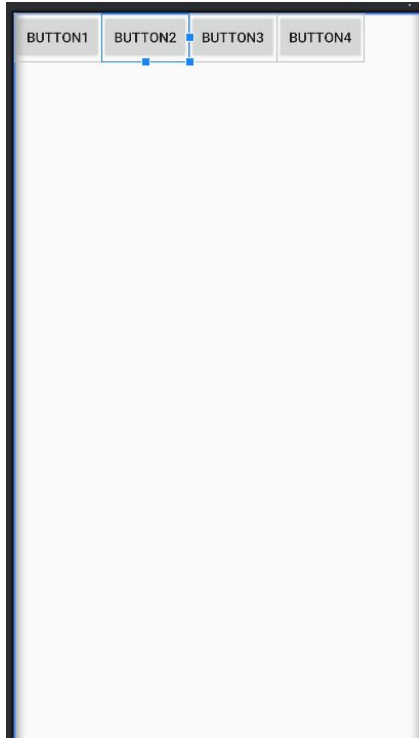
- **Linear Layout:**
 - It is a viewgroup that align all children in a single direction, either **Vertical** or **Horizontal**
- **Relative Layout:**
 - It is a viewgroup that displays child views in relative positions
- **Frame Layout:**
 - It is a placeholder on screen that you can use to display single view (overlapping over other)
- **Table Layout**
 - It is a view that groups views into rows and columns
- **Constraint Layout:**
 - More power powerful version of relative layout

Linear Layout

- This view group aligns all of its children in a single direction, either vertically or horizontally.

```
<LinearLayout xmlns:android="http://schemas.android.com/apk/res/android"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:orientation="horizontal">
    <!-- Include other widget or layout tags here. These are considered
    "child views" or "children" of the linear layout -->
</LinearLayout>
```

Linear Layout



Linear Layout

- **Attributes:**

- **android:layout_width** : Declares the width of a view. Can be wrap_content or match_parent.
- **android:layout_height**: Declares the height of a view. Can be wrap_content or match_parent.
- **android:id**: Specifies a unique identifier for the view.
- **android:orientation**: Specifies the orientation of view. Can be **vertical** or **horizontal**
- **android:layout_gravity**: Defines how the view is positioned relative to its parent layout.
- **android:layout_weight**: Equally divides the view according to screen size
- **android:layout_marginLeft**: Adds space on the left side of the view.
- **android:layout_marginRight**: Adds space on the right side of the view.
- **android:layout_marginTop**: Adds space on the top side of the view.
- **android:layout_marginBottom**: Adds space on the bottom side of the view.

Relative Layout

- Relative position of the children can be described in relation to each other or to the parent
- Each view's location can be set relative to sibling components (such as to the left or below another view) or relative to the RelativeLayout area of its parent (such as aligned to the bottom, left or center).
- Default alignment is top-left



8:39 PM

Relative Layout Examples

RED

ORANGE

YELLOW

GREEN

BLUE

INDIGO

VIOLET

Enter Name

SUBMIT

Login

Enter Username

Enter Password

Login

Frame Layout

- FrameLayout is designed to block out an area on the screen to display a single item.
- In simple words, in frame layout views are added on top of previous views.
- To add multiple children to frame layout you can use `layout_gravity` attribute to set their alignment

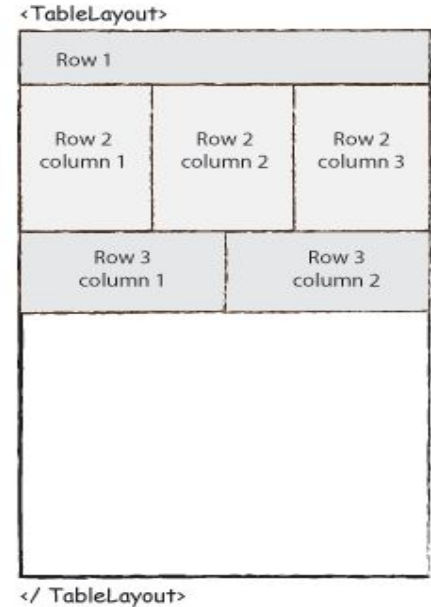
FrameWithoutGravity

abhiAndroid

abhiAndroid

Table Layout

- In Android, Table Layout is used to arrange the group of views into rows and columns.
- Table Layout containers do not display a border line for their columns, rows or cells.





TableLayoutDemo		
Roll No.	Name	Marks
101	Manish	888
102	James	878
TableLayoutDemo		
androidhub4you.com		

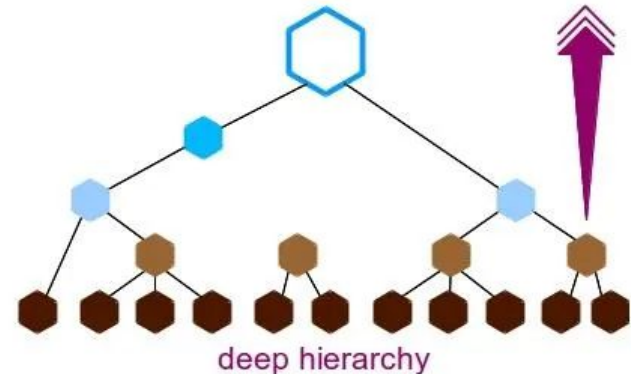
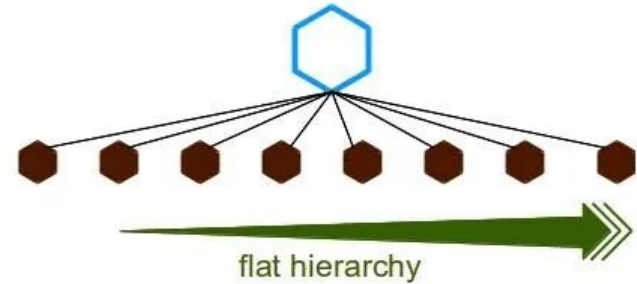
- For building a row in a table we will use the **<TableRow>** element. Table row objects are the child views of a table layout.
- Each row of the table has **zero or more cells** and each cell can hold only one view object like ImageView, TextView or any other view.
- Total width of a table is defined by its parent container
- Column can be both **stretchable and shrinkable**. If shrinkable then the width of column can be shrunk to fit the table into its parent object and if stretchable then it can expand in width to fit any extra space available.

Attributes

- **id:** id attribute is used to uniquely identify a Table Layout.
- **stretchColumns:** Stretch column attribute is used in Table Layout to change the default width of a column which is set equal to the width of the widest column but we can also stretch the columns to take up available free space by using this attribute. The value that assigned to this attribute can be a single column number or a comma delimited list of column numbers (1, 2, 3...n).
- **shrinkColumns:** Shrink column attribute is used to shrink or reduce the width of the column's. We can specify either a single column or a comma delimited list of column numbers for this attribute. The content in the specified columns word-wraps to reduce their width.
- **collapseColumns:** collapse columns attribute is used to collapse or invisible the column's of a table layout. These columns are the part of the table information but are invisible.

Constraint Layout

- Allows you to create large and complex layout with a flat view hierarchy
- More powerful version of relative layout
- Created to support drag and drop
- Every view has 2 types of anchor points
 - **Resize Handle**
 - Used to resize the view size
 - Found at corners
 - Total 4 resize handles
 - **Side Handle**
 - Circular handle
 - Used to set the top, left, bottom, right constraint
 - Total 4 side handles
- Every view has 4 edges



Simple rules

- Horizontal side handles can only be connected to horizontal edge of other view/ parent
- Vertical side handles can only be connected to Vertical edge of other view/ parent
- Need to make at least 2 connections of handle with someone else to make it constraint



